mmap 的三条路径：缺页、munmap/msync 回写、设备 mmap

(a) 缺页：第一次访问 mmap 区域

ldr/str 0x3f_ef..x
用户态访问，PTE 不存在或只读

↓

Data Abort (EC=0x24)
ESR.DFSC 0x04-0x0f：转换/访问/权限
ESR.WnR → FAULT_FLAG_WRITE
EC=0x20 指令缺页 → FAULT_FLAG_INSTR

↓

usertrap → user_page_fault
far < 最低 VMA → 原来的崩溃报告
intr_on()（读文件会睡眠）

↓

handle_mm_fault(vm, far, flags)
down_read(mmap_lock)
find_vma；不在 VMA / 权限不够
→ SIGSEGV，杀进程
copyin/out 来的：只 trylock

① PTE 已存在 + 写 + 只读
共享文件页第一次被写：AP 改成 RW = 标记“脏”，flush_tlb

② VM_PFNMAP（设备）
vm_ops->fault(vma, &vmf) 给出 pfn；PTE 加 PTE_SPECIAL
（/dev/mem、fb0 在 mmap 时已装好，一般不会走到这里）

③ VM_SHARED → shared_page_get(key, index)
pcache 命中 → mapcount++
未命中 → kalloc，放锁读文件 file_fill_page()
（匿名则清零），再加锁插入；输了竞争就丢掉自己的
文件页先装只读（干净），本次是写才装 RW

④ 私有 → kalloc
文件：file_fill_page = ireadlock + f_op->read_at
整页都在 EOF 之后 → SIGBUS；尾部补 0
匿名：清零

安装：acquire(vm->lock); walk(alloc)
已被别的线程装好 → 退回页；否则写 PTE + flush_tlb
up_read → eret 重新执行那条指令

内核替用户访问：copyin / copyout
sys_read/write/pread/pwrite 先 fault_in_user_range()
在拿 inode 锁之前把缓冲区的页都缺页进来
copyout 遇到不存在/只读页 → uva2ka_fault()
持有自旋锁（如 pipewrite）→ 直接失败，不睡眠
否则 handle_mm_fault(FAULT_FLAG_KERNEL)：只 trylock
mmap_lock，避免和 munmap 回写形成 ABBA 死锁
PTE_SPECIAL 且内核直映不覆盖（fb、MMIO）→ EFAULT
fetchaddr 不再限 [0, sz)：argv 可放在 mmap 区

fork：dup_mmap(old, new)
fork 先 down_read(mmap_lock)，再 allocproc
复制每个 VMA：filedup / shm_get / vm_ops->open
私有页：拷贝一份；共享页：pcache mapcount++，PTE 照抄
设备帧：PTE 照抄；失败 → mmap_abort 不睡眠地回滚
CLONE_VM（线程、vfork）：共享 vmspace，不复制

exit / exec：exit_mmap(vm)
exit/线程退出：vmspace_leave()，users 减到 0 才拆
exec：私有 vm 提交前拆；vfork 子进程不拆父的 VMA
逐个 VMA：zap_range(回写) → close → fileclose

(b) munmap / msync：共享文件页写回

munmap(addr, len)
down_write(mmap_lock)
__do_munmap：
中间挖洞 → 拆成两个 VMA
头/尾 → 缩短 vm_start/vm_end

→

zap_range 第 1 步：回写
逐页看 PTE：可写(AP=RW) = 脏
writeback_page：file_write_at(user_src=0)
只写到 i_size，不扩展文件
这一步会睡眠，所以不拿 vm->lock

→

zap_range 第 2 步：拆映射
acquire(vm->lock)，每批 32 个 PTE 清 0
flush_tlb（tlbi vmalle1is，所有核）
release 之后才释放页：
私有 kfree / 共享 pcache_put

→

整段都拆完 → vma_free
vm_ops->close（video0 计数--）
fileclose(vm_file)
shm_put：最后一个 VMA 释放共享匿名页

msync(addr, len, MS_SYNC)：down_read；对每个脏 PTE 先改回只读并 flush_tlb，再 write_at 回文件
顺序很重要：先变“干净”再写，并发的写会重新缺页、重新变脏，不会丢。之后 read() 能看到 mmap 里写的数据。

(c) 设备 mmap：mmap(0, len, PROT_READ|PROT_WRITE, MAP_SHARED, fd=/dev/fb0, 0)

sys_mmap
MAP_ANONYMOUS 忽略 fd
否则 fget(fd)
→ do_mmap(f, ...)

→

do_mmap
检查 prot/flags/off 对齐
down_write(mmap_lock)
get_unmapped_area / MAP_FIXED
分配 VMA，filedup(f)

→

f->f_op->mmap(f, vma)
fb_mmap：检查 pgoff/长度
mem_mmap：拒绝 PA < PHYSTOP
cammmap：只读、VM_NOFILEREF
zero：变成匿名映射

→

remap_pfn_range()
vm_flags |= VM_IO|VM_PFNMAP
vm_page_prot = 内存类型
立即写满所有 PTE
| PTE_SPECIAL，flush_tlb

→

vma_link → 返回地址
普通文件 f_op->mmap 为 0：
走通用路径，PTE 等缺页时再装
内存类型：MMIO=Device-nGnRnE，
fb/DMA=Normal-NC，与内核别名一致

测试：user/mmaptest.c 覆盖以上所有路径（QEMU raspi3b：49 项 PASS）；mmaptest fb 只画彩条。
已知限制：没有 COW（fork 复制私有页）；write() 不会更新已映射的共享页，需要 msync/munmap 之后两边才一致；没有 mprotect/mremap。